



Unidad 2. Primeros pasos con Java

El ecosistema Java (JDK, JVM, Bytecode), instalación y configuración de IntelliJ IDEA, estructura canónica de un programa, tipos de datos primitivos, variables, constantes, operadores, clases estándar (String, Math, java.time, BigDecimal), sentencias condicionales y entrada/salida por consola.



MÓDULO PROFESIONAL

Programación (0485) — 1º DAM

DURACIÓN ESTIMADA

24 horas lectivas

RESULTADOS DE APRENDIZAJE

RA1, RA4 (Estructura de programas y tipos de datos)

ENTORNO OFICIAL

IntelliJ IDEA / Java 25 LTS

¡Enhorabuena por haber superado la Unidad 1! Ya dominas los cimientos del pensamiento algorítmico, la representación mediante diagramas de flujo ANSI y el diseño estructurado con pseudocódigo. Ahora ha llegado el momento más esperado por todo estudiante de desarrollo de software: **traducir esa lógica conceptual a un lenguaje de programación real, industrial y de propósito general.**

En esta unidad nos sumergiremos en **Java** (en su versión estándar moderna **Java 25 LTS**, manteniendo compatibilidad plena con Java 21 LTS), uno de los lenguajes más robustos, seguros y demandados del panorama tecnológico mundial. Utilizaremos como entorno de trabajo oficial **IntelliJ IDEA**, la herramienta de referencia en la industria del desarrollo de software profesional.

Aprenderás cómo viaja tu código desde el editor hasta los transistores del procesador a través de la Máquina Virtual de Java (JVM), dominarás los 8 tipos primitivos de datos, comprenderás cómo manipular texto, números y fechas con precisión matemática, interactuarás con el usuario mediante la consola y controlarás el flujo de ejecución mediante sentencias condicionales clásicas y modernas.

Recursos y Descargas en PDF de la Unidad 2

Puedes estudiar esta unidad tanto en la plataforma web como en formato documento digital/impreso:

-  [Descargar Teoría Completa de la Unidad 2 en PDF](#) (41 páginas)
-  **Boletín Inicial:** [Descargar Propuestos \(PDF\)](#) · [Descargar Resueltos \(PDF\)](#)
-  **Boletín Intermedio:** [Descargar Propuestos \(PDF\)](#) · [Descargar Resueltos \(PDF\)](#)
-  **Retos y Extras:** [Descargar Retos \(PDF\)](#)

1. El Ecosistema de la Plataforma Java

1.1 Breve Historia y Filosofía de Diseño

Java nació a principios de los años 90 en **Sun Microsystems**, liderado por el ingeniero **James Gosling**. Inicialmente bautizado como *Oak* (por un roble que Gosling veía desde la ventana de su despacho), el proyecto buscaba crear un lenguaje para programar dispositivos electrónicos inteligentes (el proyecto *Star7*). Posteriormente, con el auge de Internet y la World Wide Web en 1995, el lenguaje fue renombrado como **Java** (en homenaje al café originario de la isla de Java en Indonesia) y presentado al mundo con una promesa revolucionaria:

WORA (Write Once, Run Anywhere)

“Escribe tu código una sola vez y ejecútalo en cualquier lugar.”

A diferencia de lenguajes como C o C++, cuyos programas debían recompilarse específicamente para cada sistema operativo y procesador (generando binarios dependientes de la arquitectura x86, ARM, etc.), Java introdujo un nivel de abstracción intermedio: la **Máquina Virtual de Java (JVM)**.

En 2010, **Oracle Corporation** adquirió Sun Microsystems, asumiendo la administración y liderazgo del ecosistema Java, junto con el proceso comunitario abierto **JCP (Java Community Process)** y el proyecto de código abierto **OpenJDK**.

Duke: La Mascota Oficial de Java

La simpática criatura con forma de triángulo blanco y nariz roja que acompaña a la documentación de Java se llama **Duke**. Fue diseñada originalmente por Joe Palrang como el agente interactivo que guiaba al usuario en el prototipo doméstico *Star7*. Hoy en día es el embajador de marca de la comunidad Java en todo el mundo.

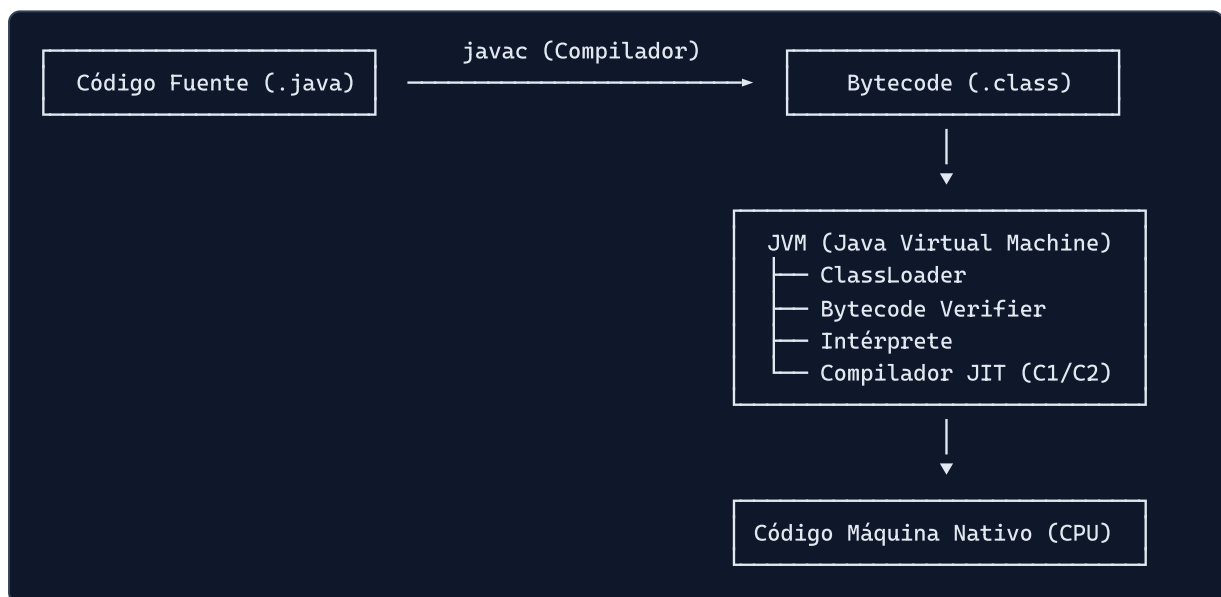
1.2 Tipología de Aplicaciones en Java

Gracias a su versatilidad y rendimiento, Java no se limita a un único entorno, sino que abarca prácticamente todas las ramas de la informática actual:

1. **Aplicaciones de Consola (CLI):** Programas basados en texto que interactúan mediante la terminal y flujos estándar (`System.in` , `System.out`), ideales para scripts, herramientas de administración y aprendizaje inicial.
2. **Aplicaciones de Escritorio con Interfaz Gráfica (GUI):** Software para sistemas operativos de usuario final creado mediante librerías gráficas como Swing o la moderna **JavaFX**.
3. **Aplicaciones Web y Servicios Backend:** La columna vertebral de la banca, el comercio electrónico y las grandes plataformas (Netflix, Amazon, Twitter). Emplea tecnologías como Servlets, Jakarta EE (anteriormente Java EE) y frameworks líderes como **Spring Boot** y Quarkus.
4. **Aplicaciones Móviles:** El sistema operativo **Android** adoptó Java desde sus inicios como lenguaje oficial para el desarrollo de apps nativas.
5. **Sistemas Embebidos e Internet de las Cosas (IoT):** Dispositivos médicos, cajeros automáticos, tarjetas SIM y sensores industriales que ejecutan entornos reducidos de Java.

1.3 La Arquitectura Técnica: Código Fuente, Bytecode y JVM

Para comprender qué ocurre cuando pulsamos el botón “Run” en nuestro entorno, debemos distinguir con total precisión las tres etapas del ciclo de ejecución:



1. **Código Fuente (`.java`):** Es el texto en lenguaje Java comprensible para los seres humanos.
2. **Compilador (`javac`):** No traduce el código fuente a lenguaje máquina directo de la CPU, sino a un código intermedio universal denominado **Bytecode** (almacenado en archivos `.class`). El Bytecode es independiente del procesador y del sistema operativo.
3. **Máquina Virtual de Java (`JVM`):** Es el software instalado en el sistema operativo del cliente que interpreta y ejecuta el Bytecode. Cada plataforma (Windows, macOS, Linux, Android) tiene su propia versión de la JVM adaptada a su hardware, pero todas interpretan exactamente el mismo Bytecode.
4. **Compilador JIT (Just-In-Time):** Para evitar la lentitud clásica de los intérpretes puros, las JVM modernas detectan las partes del código que se ejecutan repetidamente (*hot spots*) y las compilan dinámicamente sobre la marcha a código máquina nativo del procesador, logrando un rendimiento cercano al de C++.

5. **Recolector de Basura (*Garbage Collector*):** Un proceso en segundo plano de la JVM que rastrea los objetos en la memoria RAM que ya no se usan y libera su espacio automáticamente, eliminando de raíz las fugas de memoria (*memory leaks*) que plagaban a los lenguajes tradicionales.

1.4 La Tríada Fundamental: JDK, JRE y JVM

Es muy frecuente que los estudiantes principiantes confundan estas tres siglas esenciales:

Sigla	Significado	¿Qué incluye?	¿Para quién es?
JVM	<i>Java Virtual Machine</i>	Motor de ejecución, intérprete, compilador JIT y Garbage Collector.	La base técnica de todo el sistema.
JRE	<i>Java Runtime Environment</i>	La JVM + las librerías estándar de clases de Java (<code>rt.jar</code> / módulos).	Usuarios finales que solo quieren ejecutar programas ya compilados.
JDK	<i>Java Development Kit</i>	El JRE completo + herramientas de desarrollo (<code>javac</code> , <code>javadoc</code> , depuradores, empaquetador <code>jar</code>).	Desarrolladores de software . Es lo que nosotros instalaremos obligatoriamente.



1.5 Versiones de Java y Filosofía LTS

Desde 2018, Java adoptó un ritmo de lanzamientos semestral (cada 6 meses, en marzo y septiembre, se publica una nueva versión del lenguaje). Para garantizar la estabilidad industrial sin obligar a las empresas a migrar de versión cada medio año, Oracle introdujo la distinción entre versiones estándar y **LTS (*Long-Term Support*)**:

- **Versiones No-LTS (rápidas):** Tienen soporte oficial de solo 6 meses y se utilizan como banco de pruebas para novedades.
- **Versiones LTS (Soporte Extendido):** Se publican cada dos años y cuentan con un mínimo de 5 a 8 años de parches de seguridad y estabilidad empresarial.
 - **Java 8 LTS:** El hito de la programación funcional (Lambdas y Streams).
 - **Java 11 LTS:** Sistema de módulos e inferencia local con `var`.
 - **Java 17 LTS:** Registros (*records*), clases selladas y nuevo `switch`.
 - **Java 21 LTS:** Gran salto con hilos virtuales (*Virtual Threads*), pattern matching exhaustivo y máxima optimización.
- **Java 25 LTS (La versión LTS más reciente):** El estándar LTS más actual de la plataforma Java (publicado en septiembre de 2025), consolidando las características modernas, estabilidad empresarial y optimizaciones de la máquina virtual.

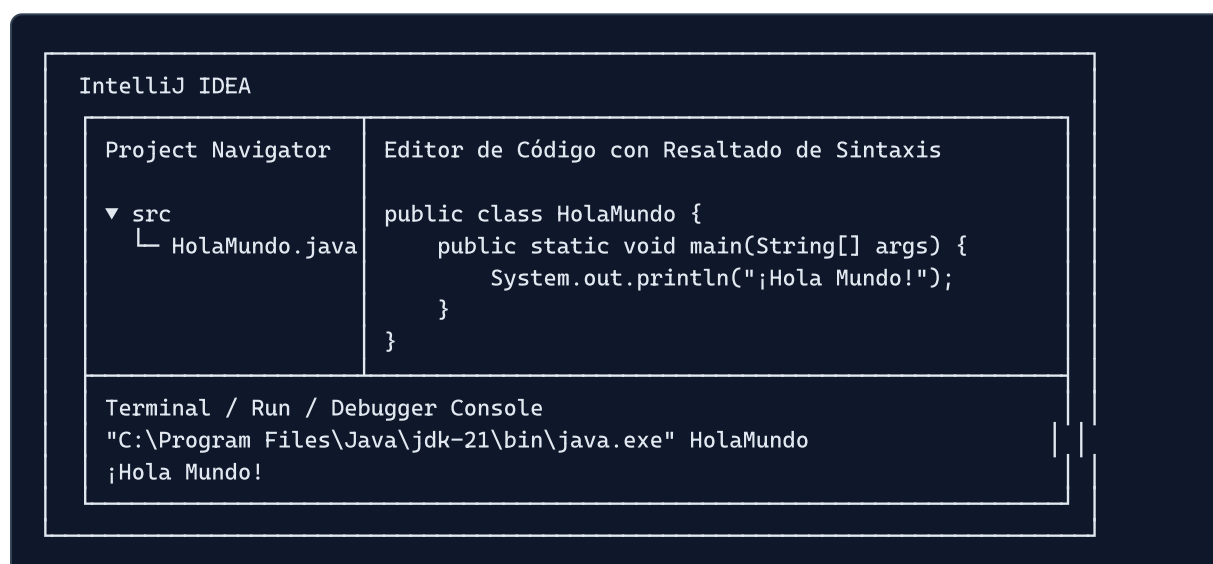
OpenJDK vs. Oracle JDK

- **OpenJDK:** La implementación oficial de código abierto y libre distribución con licencia GNU GPL v2. Es el estándar sobre el que se basan los compiladores modernos.
- **Oracle JDK:** Distribución comercial de Oracle, con condiciones específicas de licencia para entornos productivos corporativos.
- **Distribuciones comunitarias certificadas:** Existen compilaciones 100% gratuitas, de alto rendimiento y código abierto muy populares como **Eclipse Temurin (Adoptium)**, **Amazon Corretto** o **Azul Zulu**.

2. El Entorno de Desarrollo Oficial: IntelliJ IDEA

En el ciclo de DAM profesional, no escribimos código en editores de texto planos sin asistencia. Utilizamos un **Entorno Integrado de Desarrollo (IDE - Integrated Development Environment)**.

El IDE oficial para todas las explicaciones, prácticas y proyectos de este curso es **IntelliJ IDEA** (de JetBrains), la herramienta más potente, inteligente y ampliamente adoptada en el sector empresarial de Java.



2.1 Conceptos de Sistema Operativo: PATH, JAVA_HOME y CLASSPATH

Aunque IntelliJ IDEA automatiza la detección del JDK, cualquier técnico superior en informática debe conocer los tres mecanismos con los que el sistema operativo localiza las herramientas de Java:

1. **PATH**: Es una variable de entorno del sistema que contiene una lista de directorios donde el sistema operativo busca programas ejecutables cuando se invoca su nombre en la consola (como `javac` o `java`). Para que funcionen desde cualquier carpeta, se debe añadir al PATH la ruta a la subcarpeta `bin` del JDK (por ejemplo, `C:\Program Files\Java\jdk-21\bin` en Windows o `/usr/lib/jvm/jdk-21/bin` en Linux).
2. **JAVA_HOME**: Variable de entorno utilizada por servidores de aplicaciones (Tomcat, WildFly) y herramientas de compilación (Maven, Gradle) para saber exactamente en qué carpeta raíz reside el JDK principal del sistema.
3. **CLASSPATH**: Variable que indica a la JVM en qué directorios o librerías empaquetadas (`.jar`) debe buscar las clases que necesita un programa para funcionar. Por defecto, siempre incluye el directorio actual representado por un punto (`.`).

IntelliJ IDEA se encarga de todo esto por ti

En el pasado, los desarrolladores tenían que pelear manualmente configurando estas variables en el registro de Windows o en el fichero `/etc/environment` de Linux. **IntelliJ IDEA gestiona todo esto de forma autónoma:** detecta automáticamente los JDKs instalados en tu equipo o te permite descargar uno nuevo con un simple clic desde su propia interfaz.

2.2 Creación de un Nuevo Proyecto en IntelliJ IDEA

Para crear tu primer proyecto profesional en IntelliJ IDEA Community Edition:

1. Abre IntelliJ IDEA y en la pantalla de bienvenida pulsa sobre **New Project** (o desde el menú: **File** → **New** → **Project...**).
2. En el panel lateral izquierdo, selecciona **Java**.
3. En el campo **Name**, escribe el nombre del proyecto (por ejemplo, `Unidad02_PrimerosPasos`).
4. En **Location**, elige la carpeta de tu disco donde se guardará tu código.
5. En el desplegable **JDK**, selecciona tu versión instalada de **Java 25** (o **Java 21**). Si no aparece ninguna instalada, pulsa sobre **Download JDK...** y elige el proveedor *Eclipse Temurin* o *Amazon Corretto*: IntelliJ lo descargará e instalará automáticamente.
6. Marca la casilla **Add sample code** si deseas que IntelliJ cree automáticamente un archivo de ejemplo con el método `main`.
7. Pulsa **Create**.

Anatomía del Proyecto en el Disco

IntelliJ organiza los archivos de trabajo en una estructura estricta:

- `.idea/`: Carpeta oculta donde IntelliJ guarda las preferencias del proyecto, módulos y configuraciones de ejecución (no se debe modificar a mano).
- `src/` (**Source**): La carpeta más importante. Aquí residen exclusivamente nuestros archivos de código fuente con extensión `.java`.
- `out/` (o `target/`): Carpeta donde IntelliJ almacena automáticamente los archivos binarios compilados `.class` listos para ser interpretados por la JVM.

2.3 Atajos de Teclado y Plantillas Vivas (Live Templates)

La velocidad y comodidad con la que un programador escribe código en IntelliJ IDEA se debe a sus plantillas automáticas:

Plantilla / Atajo	Acción que realiza	Resultado generado
Escribir <code>main</code> o <code>psvm</code> + Tab	Genera el punto de entrada de la aplicación.	<code>public static void main(String[] args) { }</code>
Escribir <code>sout</code> + Tab	Imprime texto por consola con salto de línea.	<code>System.out.println();</code>
Escribir <code>souf</code> + Tab	Imprime texto con formato enriquecido.	<code>System.out.printf("");</code>
Ctrl + Alt + L	Reformat Code: Limpia e indenta todo el archivo con formato perfecto.	Código limpio y alineado automáticamente.
Shift + F10	Run: Compila y ejecuta el programa actual.	Abre la consola de salida.
Shift + F9	Debug: Inicia la ejecución con depurador paso a paso.	Abre el panel de inspección de variables.
Alt + Intro	Show Context Actions: Soluciona errores y sugiere mejoras.	Corrección guiada de código.

3. Anatomía de un Programa en Java

Observemos el programa clásico “Hola Mundo” estructurado conforme a las mejores prácticas de la ingeniería del software:

```
package es.iesperenxisa.primerospasos;

/**
 * Clase principal para ilustrar la anatomía de un programa en Java.
 * @author Docente DAM
 * @version 1.0
 */
public class HolaMundo {

    public static void main(String[] args) {
        // Imprime un saludo por la consola estándar del sistema
        System.out.println("¡Hola, programadores de DAM!");
    }
}
```

3.1 Desglose Riguroso del Código

1. `package es.iesperenxisa.primerospasos;`

Indica el **paquete** (espacio de nombres) al que pertenece la clase. Los paquetes evitan colisiones de nombres entre clases y organizan el proyecto en carpetas físicas en el disco. La convención internacional de Java exige utilizar el nombre de dominio de la organización en orden inverso (por ejemplo, `es.iesperenxisa`).

2. `public class HolaMundo`

En Java, **todo el código debe residir obligatoriamente dentro de una clase**. No pueden existir funciones o variables “sueltas” en el archivo.

- **Regla estricta del compilador:** Si una clase se declara con el modificador `public`, el archivo en el disco **debe llamarse exactamente igual que la clase** y con extensión `.java` (`HolaMundo.java`). Además, solo puede existir una única clase pública por cada archivo físico.

3. `public static void main(String[] args)`

Es el **punto de entrada universal (Entry Point)** de cualquier aplicación ejecutable en Java. Cuando le pedimos a la JVM que lance nuestra aplicación, busca exactamente este método:

- `public`: Modificador de visibilidad que permite a la JVM invocar el método desde fuera de la clase.
- `static`: Permite que la JVM invoque el método sin necesidad de crear previamente un objeto de la clase `HolaMundo` en memoria.
- `void`: Indica que el método realiza una tarea pero no devuelve ningún valor como resultado de su ejecución.
- `main`: Nombre reservado que busca el lanzador de Java.
- `String[] args`: Parámetro que representa un array de cadenas de texto para capturar argumentos pasados por línea de comandos al arrancar el programa.

4. `System.out.println( ... )`

- `System`: Es una clase predefinida en el paquete nuclear de Java (`java.lang`).
- `out`: Es un objeto estático dentro de `System` que representa el flujo de salida estándar (la consola de texto).
- `println()`: Es el método que escribe el contenido entre paréntesis en la pantalla y añade automáticamente un salto de línea al final. Su homólogo `System.out.print()` muestra el contenido pero mantiene el cursor en la misma línea.

3.2 Tipología de Comentarios y Generación con JavaDoc

Java soporta tres modalidades de comentarios:

```
// 1. Comentario de una sola línea: ideal para notas breves y directas.

/*
  2. Comentario de bloque o multilínea:
  útil para explicaciones extensas o para
  desactivar temporalmente fragmentos de código.
*/

/**
 * 3. Comentario de documentación técnica (JavaDoc).
 * Describe el propósito de una clase o método y es procesado
 * por la herramienta javadoc para generar páginas web HTML de documentación.
 * @param args Argumentos de la línea de comandos
 */
```

Etiquetas JavaDoc más Comunes:

- `@author Nombre`: Especifica el autor del componente software.
- `@version 1.0`: Indica la versión actual del código.
- `@param nombre Parametro`: Explica qué representa cada parámetro recibido por un método.
- `@return Descripción`: Detalla qué valor devuelve el método al finalizar.
- `@throws TipoExcepcion`: Documenta qué errores puede lanzar el método si algo falla.

En IntelliJ IDEA, puedes generar la documentación HTML completa de tu proyecto navegando a:

Tools → Generate JavaDoc...

3.3 Compilación y Ejecución Manual desde la Terminal

Aunque el IDE automatiza el proceso, un desarrollador profesional debe comprender cómo interactuar directamente con el compilador desde la consola de comandos de Windows o Linux:

Ventana de terminal

```
# 1. Compilación: Transforma el archivo fuente en Bytecode
# El flag -d indica la carpeta destino donde dejar los binarios compilados
javac -d out/production/Unidad02 src/es/iesperenxisa/primerospasos/HolaMundo.java

# 2. Ejecución: Se lanza la JVM indicando el classpath (-cp) y el nombre canónico de la clase
java -cp out/production/Unidad02 es.iesperenxisa.primerospasos.HolaMundo

# 3. Novedad de Java moderno (Java 11+): Ejecución directa sin compilar a disco
java src/HolaMundo.java
```

4. Elementos Básicos del Lenguaje: Identificadores, Variables y Constantes

Para construir cualquier algoritmo en un lenguaje de programación necesitamos mecanismos para nombrar entidades, almacenar datos temporales en la memoria RAM y garantizar que ciertos valores permanezcan inmutables a lo largo del tiempo.

4.1 Identificadores y Convenciones de Nomenclatura

Un **identificador** es el nombre que el programador asigna a un elemento del código: un paquete, una clase, una variable, una constante o un método.

Reglas Léxicas Obligatorias del Compilador

Si vulneras alguna de estas reglas, el compilador `javac` generará un error de sintaxis y el programa no compilará:

1. Debe comenzar por una letra Unicode (del alfabeto latino básico), un carácter de subrayado (`_`) o el signo de dólar (`$`). **Nunca puede comenzar por un dígito.**
2. Los caracteres posteriores pueden ser letras, dígitos (0-9), subrayados (`_`) o signos de dólar (`$`).
3. Java distingue de manera estricta entre mayúsculas y minúsculas (*case-sensitive*). Por ejemplo, `edad`, `Edad` y `EDAD` son tres identificadores totalmente distintos e independientes.
4. No puede coincidir con ninguna de las **palabras reservadas** (*keywords*) del lenguaje Java (como `class`, `public`, `static`, `void`, `int`, `if`, `while`, `return`, etc.), ni con los literales reservados `true`, `false` y `null`.
5. No puede contener espacios en blanco ni operadores matemáticos o signos de puntuación (`+`, `-`, `*`, `/`, `@`, `#`, `.`, `,`).

Atención con tildes y caracteres no ASCII

Aunque el compilador de Java soporta caracteres Unicode completos en identificadores (lo que permite declarar variables como `año` o `número`), **en el desarrollo profesional está terminantemente desaconsejado**. Utilizar caracteres fuera del estándar ASCII (como la `ñ`, tildes, diéresis o caracteres cirílicos/asiáticos) genera graves problemas de compatibilidad al compartir código entre diferentes sistemas operativos (Windows, Linux, macOS), servidores de integración continua y herramientas de control de versiones como Git. Se debe utilizar siempre el alfabeto inglés sin tildes: `anio` o `year`, `numero` o `number`.

Convenciones de la Comunidad Java (Code Style de Oracle e IntelliJ)

A diferencia de las reglas del compilador (que son obligatorias), las convenciones son acuerdos universales seguidos por millones de desarrolladores para que el código sea inmediatamente legible:

Elemento	Convención de Nomenclatura	Ejemplo Recomendado	Antiejemplo a Evitar
Clases e Interfaces	PascalCase / UpperCamelCase (cada palabra empieza en mayúscula)	<code>CuentaBancaria</code> , <code>GestorPedidos</code>	<code>cuentaBancaria</code> , <code>gestor_pedidos</code>
Variables y Métodos	lowerCamelCase (primera palabra minúscula, siguientes en mayúscula)	<code>saldoActual</code> , <code>calcularInteres()</code>	<code>SaldoActual</code> , <code>calcular_interes</code>
Constantes	UPPER_SNAKE_CASE (todo mayúsculas separadas por guion bajo)	<code>PI</code> , <code>MAX_INTENTOS_LOGIN</code>	<code>maxIntentos</code> , <code>max_intentos</code>
Paquetes	Todo minúsculas en orden inverso de dominio de internet	<code>es.iesperenxisa.primerospasos</code>	<code>es.IESPerenxisa.PrimerosPasos</code>

4.2 Declaración, Inicialización y Ámbito de Variables

Una **variable** es un contenedor con nombre asociado a una posición de la memoria RAM del ordenador, capaz de albergar un dato de un tipo concreto que puede ser modificado durante la ejecución del programa.

En Java, toda variable debe ser declarada especificando obligatoriamente su **tipo de dato** y su **identificador**:

```
// 1. Declaración simple (se reserva el espacio en memoria, sin asignar valor explícito)
int edadUsuario;
double temperaturaMedia;

// 2. Inicialización posterior (asignación del primer valor)
edadUsuario = 25;
temperaturaMedia = 21.4;

// 3. Declaración e inicialización combinada en una sola línea (recomendado)
int puntos = 100;
boolean usuarioActivo = true;

// 4. Declaración múltiple del mismo tipo (permitido por sintaxis, pero desaconsejado)
int x = 0, y = 10, z = 20;
```

Ámbito (Scope) y el Peligro de Variables Locales no Inicializadas

El ámbito de una variable es la región del código fuente comprendida entre las llaves `{ ... }` donde fue declarada. Fuera de ese bloque, la variable deja de existir para el compilador y para la memoria:

```
public void calcular() {
    int total = 10; // Variable local accesible en todo el método

    if (total > 5) {
        int descuento = 2; // Variable local al bloque if
        total -= descuento;
    } // Aquí 'descuento' se destruye y ya no es accesible

    // System.out.println(descuento); // ¡ERROR de compilación! 'descuento' no existe aquí
}
```

Regla de Oro en Java: Las variables locales NO tienen valor por defecto

A diferencia de los atributos o campos de una clase (que la JVM inicializa automáticamente a `0`, `0.0`, `false` o `null`), **las variables locales declaradas dentro de un método NO reciben ningún valor por defecto**. Si intentas leer o utilizar una variable local sin haberle asignado un valor previamente, el compilador detendrá el proceso arrojando el error:

```
java: variable edadUsuario might not have been initialized
```

4.3 Constantes con la Palabra Clave `final`

Una **constante** es una variable cuyo valor no puede ser alterado una vez que ha sido asignado por primera vez. En Java se definen anteponiendo el modificador `final`:

```
public class ConstantesDemo {
    public static void main(String[] args) {
        final double TASA_IVA = 0.21;
        final int DIAS_SEMANA = 7;
        final String NOMBRE_APLICACION = "GestorDAM";

        // TASA_IVA = 0.10; // ¡ERROR de compilación! Cannot assign a value to final variable
    }
}
```

Ventajas de utilizar constantes:

1. **Evitan los “números mágicos”:** Un código con valores como `3.14159` o `86400` repartidos por el código es difícil de entender y mantener. Al bautizarlos como `final double PI` o `final int SEGUNDOS_POR_DIA`, el código se autoexplica.
2. **Seguridad e Integridad:** Garantizan que parámetros críticos del sistema (tasas impositivas, límites de memoria, puertos de red) no puedan ser sobrescritos por accidente por otro método.
3. **Optimizaciones del compilador JIT:** Al saber que el valor jamás cambiará, la JVM puede sustituir la variable directamente por el valor literal en tiempo de ejecución (*inlining*), mejorando el rendimiento.

4.4 Inferencia de Tipos en Variables Locales con `var` (Java 10+)

A partir de Java 10 (2018), el lenguaje incorporó la palabra reservada contextual `var`, permitiendo al compilador inferir automáticamente el tipo de la variable a partir del valor de la expresión a su derecha:

```
// Declaración tradicional con tipo explícito
String mensaje = "Hola Mundo";
int contador = 42;
double precio = 99.95;

// Declaración moderna con inferencia de tipos (var)
var mensajeMod = "Hola Mundo"; // El compilador infiere String
var contadorMod = 42;           // El compilador infiere int
var precioMod = 99.95;          // El compilador infiere double
```

Reglas Estrictas de `var`:

1. **Java sigue siendo un lenguaje fuertemente tipado:** `var` **NO** convierte a Java en JavaScript o Python. El tipo de la variable se fija irrevocablemente durante la compilación. Una vez que `var contadorMod = 42` es compilado como `int`, intentar asignarle un texto (`contadorMod = "Texto"`) causará un error de compilación.
2. **Solo para variables locales con inicializador:** No se puede usar `var` en atributos de clase, ni en parámetros de métodos, ni en tipos de retorno, ni sin asignar un valor en la misma línea (`var x;` es ilegal).
3. **Buenas prácticas:** Usa `var` cuando el tipo sea evidente y reduzca la redundancia visual (por ejemplo al instanciar objetos complejos `var scanner = new Scanner(System.in);`). Evita su uso cuando el tipo de retorno de una función sea ambiguo y dificulte la lectura al desarrollador humano.

5. El Sistema de Tipos de Java: Los 8 Tipos Primitivos

Java es un lenguaje de **tipado estático y fuerte**:

- **Estático:** El tipo de cada variable se comprueba y valida en tiempo de compilación.
- **Fuerte:** No se permiten operaciones entre tipos incompatibles sin una conversión explícita.

En el nivel más bajo de la arquitectura de memoria de Java residen los **8 tipos primitivos**, diseñados para ofrecer el máximo rendimiento al mapearse directamente sobre las instrucciones del procesador.

```
Tipos de Datos en Java
├── Primitivos (8 tipos básicos almacenados directamente por valor en Stack)
│   ├── Numéricos Enteros (byte, short, int, long)
│   ├── Numéricos de Coma Flotante (float, double)
│   ├── Carácter (char)
│   └── Booleano (boolean)
└── Referenciados / Objetos (almacenados en Heap, referenciados por puntero)
    ├── Clases (String, Scanner, Integer, etc.)
    ├── Interfaces
    ├── Arrays (vectores y matrices)
    └── Records y Enums
```

5.1 Tabla Comparativa Completa de Tipos Primitivos

Tipo	Categoría	Tamaño	Rango de Valores	Literal / Sufijo	Valor Defecto (en clases)
<code>byte</code>	Entero	8 bits (1 byte)	-128 a 127	<code>(byte) 10</code>	<code>0</code>
<code>short</code>	Entero	16 bits (2 bytes)	-32.768 a 32.767	<code>(short) 3000</code>	<code>0</code>
<code>int</code>	Entero	32 bits (4 bytes)	-2.147.483.648 a 2.147.483.647 (-2^{31} a $2^{31}-1$)	<code>100</code>	<code>0</code>
<code>long</code>	Entero	64 bits (8 bytes)	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	<code>100L</code> o <code>100l</code>	<code>0L</code>
<code>float</code>	Flotante	32 bits (4 bytes)	$\pm 1.4 \times 10^{-45}$ a $\pm 3.4028235 \times 10^{38}$ (~7 dígitos de precisión)	<code>3.14f</code> o <code>3.14F</code>	<code>0.0f</code>
<code>double</code>	Flotante	64 bits (8 bytes)	$\pm 4.9 \times 10^{-324}$ a $\pm 1.7976931348623157 \times 10^{308}$ (~16 dígitos)	<code>3.14</code> o <code>3.14d</code>	<code>0.0d</code>
<code>char</code>	Carácter	16 bits (2 bytes)	Código Unicode UTF-16: <code>\u0000</code> (0) a <code>\uffff</code> (65.535)	<code>'A'</code> , <code>'\n'</code> , <code>'\u0041'</code>	<code>'\u0000'</code>
<code>boolean</code>	Lógico	JVM dependiente (~8 bits)	<code>true</code> o <code>false</code>	<code>true</code> , <code>false</code>	<code>false</code>

Aspectos Críticos a Recordar:

- Los enteros literales son siempre `int` por defecto:** Si escribes `3000000000` (3 mil millones), el compilador dará error porque supera el rango de `int`. Para convertirlo en un literal de tipo `long`, debes añadir obligatoriamente la letra `L` al final: `3000000000L` (utiliza siempre `L` mayúscula, ya que `l` minúscula se confunde visualmente con el número `1`).
- Los decimales literales son siempre `double` por defecto:** Si escribes `float f = 3.14;`, el compilador emitirá un error indicando pérdida de precisión (un `double` no cabe automáticamente en un `float`). Debes indicar `3.14f`.
- El tipo `boolean` es puro:** En Java, a diferencia de C/C++, un **booleano NUNCA equivale a 0 o 1**. `if (1) { ... }` o `boolean flag = 0;` son errores de compilación directos.
- El tipo `char` representa números de 16 bits sin signo:** Los caracteres se delimitan con comillas simples (`'A'`), mientras que los textos se delimitan con comillas dobles (`"A"`). Un `char` puede operarse aritméticamente: `'A' + 1` da el número entero `66` (código ASCII de `'B'`).

5.2 Notación de Literales Numéricos y Formatos Alternativos

Java admite representar literales numéricos enteros en distintas bases numéricas y con separadores de legibilidad:

```
// 1. Sistema Decimal (base 10 - habitual)
int decimal = 100;

// 2. Sistema Hexadecimal (base 16 - prefijo 0x o 0X)
int hex = 0x2A; // Equivale a 42 en decimal (2 * 16 + 10)
int colorBlanco = 0xFFFFFFFF;

// 3. Sistema Binario (base 2 - prefijo 0b o 0B, introducido en Java 7)
int mascaraBinaria = 0b101010; // Equivale a 42 en decimal

// 4. Sistema Octal (base 8 - prefijo 0)
// ¡CUIDADO! Un número que empieza por 0 se interpreta como octal
int octal = 052; // Equivale a 42 en decimal (5 * 8 + 2)

// 5. Separador visual con guion bajo (Java 7+)
// No altera el valor numérico, solo mejora drásticamente la legibilidad humana
long tarjetaCredito = 4552_1100_8923_4321L;
double unMillon = 1_000_000.00;
int bytesMascara = 0b1111_0000_1010_0101;
```

5.3 Desbordamiento Numérico Cíclico (*Integer Overflow*)

Los tipos primitivos enteros en Java utilizan la codificación de **complemento a dos** (*two's complement*) para representar valores positivos y negativos. ¿Qué sucede si sumamos 1 al valor máximo que puede almacenar un tipo entero?

```
byte b = 127; // Máximo valor positivo representable en 8 bits (-128 a 127)
b++;          // Sumamos 1
System.out.println(b); // Imprime: -128
```

El Desbordamiento NO lanza ninguna excepción

En Java, las operaciones aritméticas primitivas que superan los límites de rango **no detienen el programa ni lanzan errores**. Simplemente se produce un desbordamiento cíclico silencioso: el bit de signo cambia y el valor pasa al extremo negativo opuesto.

Si estás trabajando con cálculos matemáticos críticos, acumuladores financieros o conteos masivos, debes utilizar la clase `Math` con sus métodos de comprobación exacta introducidos en Java 8, tales como

`Math.addExact(a, b)` o `Math.multiplyExact(a, b)`, los cuales lanzan inmediatamente una excepción `ArithmeticException` si detectan un desbordamiento.

6. Conversión de Tipos (*Type Casting*) y Clases Envoltorio (*Wrappers*)

A menudo necesitamos transferir datos entre variables de diferente naturaleza. Dependiendo de si existe o no riesgo de pérdida de información, Java clasifica las conversiones en dos grandes categorías.

6.1 Conversión Implícita o Promoción (*Widening Casting*)

Ocurre de manera totalmente **automática y transparente** cuando se asigna un valor de un tipo con menor capacidad o rango a una variable de un tipo con mayor capacidad. Como no existe peligro de pérdida de magnitud, el compilador lo permite sin advertencias:

```
byte → short → int → long → float → double
char ↗
```

```
int numeroEntero = 100;
long numeroLargo = numeroEntero; // Automático: int (32 bits) a long (64 bits)
double numeroReal = numeroLargo; // Automático: long a double (64 bits coma
flotante)
System.out.println(numeroReal); // Imprime: 100.0
```

6.2 Conversión Explícita o Truncamiento (Narrowing Casting)

Ocurre cuando intentamos guardar un dato de mayor rango o precisión en un contenedor más pequeño. Como existe un **riesgo real de pérdida de información o truncamiento de decimales**, el compilador exige que el programador declare su intención explícitamente mediante el operador de moldeado (**tipo**):

```
double precioConIva = 99.89;
// int precioEntero = precioConIva; // ¡ERROR de compilación! Possible loss of
precision

// Casting explícito: se descartan por completo los decimales (truncamiento hacia cero)
int precioEntero = (int) precioConIva;
System.out.println(precioEntero); // Imprime: 99 (¡se han perdido los 0.89!)

// Peligro de desbordamiento en narrowing:
int valorGrande = 300;
byte valorByte = (byte) valorGrande; // 300 no cabe en 8 bits (-128 a 127)
System.out.println(valorByte); // Imprime: 44 (resultado del desbordamiento
binario)
```

6.3 Conversión entre Cadenas (String) y Tipos Numéricos

La interacción por consola, archivos o interfaces gráficas siempre se realiza mediante texto (**String**). Para realizar cálculos aritméticos con esos datos, debemos convertirlos a sus tipos primitivos equivalentes mediante los métodos de parseo de las clases envoltorio:

```
// 1. De String a Tipos Primitivos (Parseo)
String textoEdad = "25";
int edad = Integer.parseInt(textoEdad);

String textoSueldo = "1850.75";
double sueldo = Double.parseDouble(textoSueldo);

String textoActivo = "true";
boolean activo = Boolean.parseBoolean(textoActivo);

// 2. De Tipos Primitivos a String
int dias = 365;
String s1 = String.valueOf(dias); // Forma óptima y recomendada
String s2 = Integer.toString(dias); // Equivalente directo
String s3 = "" + dias; // Válido por concatenación, pero menos eficiente
```

La temida excepción **NumberFormatException**

Si intentas parsear una cadena que contiene caracteres no numéricos o espacios en blanco residuales (por ejemplo, `Integer.parseInt(" 25a ")`), la JVM detendrá el programa inmediatamente arrojando un error de tipo `java.lang.NumberFormatException`. Más adelante aprenderemos a capturarlo mediante bloques `try-catch`.

6.4 Clases Envoltorio (Wrapper Classes) y Autoboxing/Unboxing

Dado que Java es un lenguaje orientado a objetos pero los tipos primitivos no son objetos (no tienen métodos ni atributos), Java proporciona una **Clase Envoltorio** (*Wrapper*) correspondiente a cada uno de los 8 tipos primitivos dentro del paquete estándar `java.lang`:

Tipo Primitivo	Clase Envoltorio (Wrapper)	Métodos y Constantes de Gran Utilidad
<code>byte</code>	<code>Byte</code>	<code>Byte.MAX_VALUE</code> , <code>Byte.MIN_VALUE</code> , <code>Byte.parseByte()</code>
<code>short</code>	<code>Short</code>	<code>Short.MAX_VALUE</code> , <code>Short.MIN_VALUE</code>
<code>int</code>	<code>Integer</code>	<code>Integer.MAX_VALUE</code> , <code>Integer.toBinaryString()</code> , <code>Integer.compare()</code>
<code>long</code>	<code>Long</code>	<code>Long.MAX_VALUE</code> , <code>Long.parseLong()</code>
<code>float</code>	<code>Float</code>	<code>Float.isNaN()</code> , <code>Float.isInfinite()</code>
<code>double</code>	<code>Double</code>	<code>Double.POSITIVE_INFINITY</code> , <code>Double.parseDouble()</code>
<code>char</code>	<code>Character</code>	<code>Character.isDigit()</code> , <code>Character.isLetter()</code> , <code>Character.toUpperCase()</code>
<code>boolean</code>	<code>Boolean</code>	<code>Boolean.TRUE</code> , <code>Boolean.FALSE</code> , <code>Boolean.logicalAnd()</code>

Autoboxing y Unboxing Automático

Desde Java 5, el compilador realiza de forma totalmente automática la conversión entre el tipo primitivo y su correspondiente objeto envoltorio:

- **Autoboxing:** Conversión automática de un primitivo a su clase Wrapper (`int` → `Integer`).
- **Unboxing:** Conversión automática de un objeto Wrapper a su primitivo básico (`Integer` → `int`).

```
// Autoboxing: el compilador ejecuta internamente Integer.valueOf(50)
Integer objetoEntero = 50;

// Unboxing: el compilador extrae el valor primitivo con objetoEntero.intValue()
int primitivo = objetoEntero;

// Operaciones aritméticas transparentes entre wrappers y primitivos
objetoEntero = objetoEntero + 10;
```

La Trampa Mortal del Unboxing con valores `null`

Un primitivo (`int`) siempre tiene un valor numérico. Sin embargo, un objeto Wrapper (`Integer`) es una referencia y **puede valer** `null`. Si la JVM intenta realizar un unboxing sobre una referencia nula, provocará una catástrofe en tiempo de ejecución:

```
Integer puntos = null; // No tiene ningún objeto asociado
int resultado = puntos + 5; // ¡Lanza NullPointerException en tiempo de ejecución!
```

7. Operadores y Precedencia

Los operadores son símbolos especiales que indican a la CPU que realice cálculos matemáticos, comparaciones lógicas o manipulaciones de bits sobre uno o más operandos.

7.1 Operadores Aritméticos y Asignación Compuesta

Java incluye los 5 operadores aritméticos fundamentales:

```
int a = 15;
int b = 4;

int suma = a + b;           // 19
int resta = a - b;          // 11
int multiplicacion = a * b;  // 60
int divisionEntera = a / b;  // 3  (¡Atención: trunca la parte decimal!)
int modulo = a % b;          // 3  (Resto exacto de la división: 15 = 4 * 3 + 3)
```

División Entera vs División Real

Si ambos operandos son números enteros, el operador `/` efectúa una **división entera**, ignorando por completo cualquier residuo fraccionario. Para obtener un resultado con decimales, **al menos uno de los operandos debe ser de coma flotante** (`double` o `float`):

```
int x = 7;
int y = 2;
double r1 = x / y;           // Evalúa 7 / 2 = 3 → asigna 3.0 (¡Error común!)
double r2 = (double) x / y;  // Hace casting de x a 7.0 → 7.0 / 2 = 3.5 (¡Correcto!)
```

Aplicaciones Prácticas del Operador Módulo (`%`)

El operador `%` no es una simple curiosidad matemática; es una de las herramientas más utilizadas en la algoritmia diaria:

1. **Comprobar si un número es par o impar:** `if (n % 2 == 0) // Es par`
2. **Comprobar múltiplos exactos:** `if (anio % 4 == 0) // Posible bisiesto`
3. **Controlar ciclos y relojes circulares:** `hora = (hora + 1) % 24;`
4. **Descomposición de dígitos decimales:**
 - Última cifra de un número: `numero % 10`
 - Eliminar la última cifra: `numero / 10`

Operadores de Asignación Compuesta

Permiten abreviar operaciones donde una variable se modifica a sí misma:

Operador Compuesto	Expresión Equivalente	Ejemplo
<code>+=</code>	<code>a = a + b</code>	<code>saldo += ingreso;</code>
<code>-=</code>	<code>a = a - b</code>	<code>stock -= unidadesVendidas;</code>
<code>*=</code>	<code>a = a * b</code>	<code>precio *= 1.21;</code>
<code>/=</code>	<code>a = a / b</code>	<code>puntos /= 2;</code>
<code>%=</code>	<code>a = a % b</code>	<code>minutos %= 60;</code>

7.2 Operadores de Incremento y Decremento

Java proporciona los operadores unarios `++` (sumar 1) y `--` (restar 1). Pueden situarse antes de la variable (**prefijo**) o después (**postfijo**), y su comportamiento en expresiones combinadas es profundamente diferente:

```
int x = 5;
int y = 5;

// Postfijo (x++): Se utiliza primero el valor actual en la expresión, y luego se incrementa
int resultadoPost = x++;
System.out.println(resultadoPost); // Imprime: 5
System.out.println(x);             // Imprime: 6

// Prefijo (++y): Se incrementa primero el valor, y luego se utiliza en la expresión
int resultadoPre = ++y;
System.out.println(resultadoPre); // Imprime: 6
System.out.println(y);           // Imprime: 6
```

Consejo de Legibilidad

Salvo en la cabecera de bucles `for` (donde `i++` es estándar), evita incrustar operadores `++` y `--` dentro de expresiones aritméticas complejas o llamadas a métodos. Separar la operación en una línea independiente (`x++;`) hace que el código sea inmune a confusiones lógicas.

7.3 Operadores Relacionales y de Comparación

Se utilizan para comparar dos valores. El resultado de cualquier operación relacional es siempre un valor primitivo `boolean` (`true` o `false`):

Operador	Significado	Ejemplo (<code>a = 10</code> , <code>b = 20</code>)	Resultado
<code>==</code>	Igual a	<code>a == b</code>	<code>false</code>
<code>≠</code>	Distinto de	<code>a ≠ b</code>	<code>true</code>
<code>></code>	Mayor que	<code>a > b</code>	<code>false</code>
<code><</code>	Menor que	<code>a < b</code>	<code>true</code>
<code>≥</code>	Mayor o igual que	<code>a ≥ 10</code>	<code>true</code>
<code>≤</code>	Menor o igual que	<code>b ≤ 20</code>	<code>true</code>

7.4 Operadores Lógicos y Cortocircuito (*Short-Circuit Evaluation*)

Permiten combinar múltiples expresiones booleanas para formar condiciones complejas:

Operador	Nombre Lógico	Comportamiento
<code>&&</code>	AND lógico condicional	Devuelve <code>true</code> si y solo si ambos operandos son <code>true</code> . Con cortocircuito.
<code> </code>	OR lógico condicional	Devuelve <code>true</code> si al menos uno de los operandos es <code>true</code> . Con cortocircuito.
<code>!</code>	NOT (Negación)	Invierte el valor booleano: <code>!true</code> da <code>false</code> , <code>!false</code> da <code>true</code> .
<code>^</code>	XOR (O exclusivo)	Devuelve <code>true</code> si uno de los operandos es <code>true</code> y el otro <code>false</code> .
<code>&</code>	AND estricto / lógico	Evalúa siempre ambos operandos (sin cortocircuito).
<code> </code>	OR estricto / lógico	Evalúa siempre ambos operandos (sin cortocircuito).

¿Qué es la Evaluación en Cortocircuito (*Short-Circuit*)?

Los operadores `&&` y `||` son enormemente eficientes y seguros gracias a la evaluación en cortocircuito:

- En una expresión `A && B`, si `A` resulta ser `false`, el resultado global será inevitablemente `false`. Java **no se molesta en evaluar la expresión `B`**.
- En una expresión `A || B`, si `A` resulta ser `true`, el resultado global será indefectiblemente `true`. Java **omite la evaluación de `B`**.

Patrón Defensivo Fundamental en Java: El cortocircuito permite proteger nuestro programa contra fallos críticos en tiempo de ejecución:

```
String texto = null;

// Protección perfecta contra NullPointerException:
// Si texto == null es true, texto.length() JAMÁS llega a ejecutarse
if (texto != null && texto.length() > 0) {
    System.out.println("El texto no está vacío");
}

// División segura contra división por cero:
int divisor = 0;
if (divisor != 0 && (100 / divisor > 2)) {
    System.out.println("Operación válida");
}
```

Si hubieras utilizado el operador `&` en lugar de `&&`, Java habría evaluado `texto.length()` o `100 / divisor`, estrellando la aplicación con un error fatal.

7.5 Operador Condicional Ternario (`?:`)

Es el único operador ternario de Java (requiere tres operandos). Constituye una versión compacta de una sentencia `if-else` que devuelve directamente un valor evaluado:

```
variable = (condicion) ? valor_si_verdadero : valor_si_falso;
```

```
int edad = 19;
String estado = (edad ≥ 18) ? "Mayor de edad" : "Menor de edad";

// Cálculo de valor absoluto:
int valor = -45;
int absoluto = (valor < 0) ? -valor : valor;
```

7.6 Tabla Completa de Precedencia y Asociatividad de Operadores

Cuando una misma expresión contiene múltiples operadores, Java sigue un orden estricto de precedencia matemática y lógica para determinar qué operación se resuelve primero:

Precedencia	Tipo de Operador	Operadores	Asociatividad
1 (Máxima)	Acceso, llamadas, corchetes	<code>.</code> <code>[]</code> <code>()</code>	De izquierda a derecha
2	Unarios y sufijos	<code>expr++</code> <code>expr--</code>	De derecha a izquierda
3	Unarios y prefijos	<code>++expr</code> <code>--expr</code> <code>+</code> <code>-</code> <code>!</code> <code>~</code> <code>(tipo)</code>	De derecha a izquierda
4	Multiplicativos	<code>*</code> <code>/</code> <code>%</code>	De izquierda a derecha
5	Aditivos	<code>+</code> <code>-</code>	De izquierda a derecha
6	Desplazamiento de bits	<code><<</code> <code>>></code> <code>>>></code>	De izquierda a derecha
7	Relacionales	<code><</code> <code>></code> <code>≤</code> <code>≥</code> <code>instanceof</code>	De izquierda a derecha
8	Igualdad	<code>==</code> <code>≠</code>	De izquierda a derecha
9	AND a nivel de bits	<code>&</code>	De izquierda a derecha
10	XOR a nivel de bits	<code>^</code>	De izquierda a derecha
11	OR a nivel de bits	<code> </code>	De izquierda a derecha
12	AND lógico (cortocircuito)	<code>&&</code>	De izquierda a derecha
13	OR lógico (cortocircuito)	<code> </code>	De izquierda a derecha
14	Ternario	<code>?:</code>	De derecha a izquierda
15 (Mínima)	Asignación	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> etc.	De derecha a izquierda

Usa Paréntesis Sin Miedo

No intentes memorizar toda la tabla de precedencia ni obligues a otros desarrolladores a descifrar expresiones complejas como `x + y * z > w && a || b`. **Usa paréntesis deliberadamente:** clarifican la intención del algoritmo y garantizan que el orden de evaluación sea inequívoco.

8. Clases Estándar Fundamentales de la Biblioteca de Java

El Java Development Kit (JDK) incluye miles de clases listas para usar dentro del paquete `java.lang` (que se importa de forma automática e implícita en todos los ficheros `.java`) y otros paquetes estándar como `java.time` o `java.math`.

8.1 La Clase `Math`

Ubicada en `java.lang.Math`, ofrece una colección completa de métodos matemáticos y constantes estáticas:

```
// 1. Constantes matemáticas universales
double pi = Math.PI; // 3.141592653589793
double e = Math.E; // 2.718281828459045

// 2. Funciones de cálculo y raíces
double potencia = Math.pow(2, 8); // 2 elevado a 8 = 256.0
double raizCuadrada = Math.sqrt(49); // 7.0
double raizCubica = Math.cbrt(27); // 3.0
int absoluto = Math.abs(-15); // 15

// 3. Métodos de redondeo
double valor = 4.7;
long redondeo = Math.round(valor); // 5 (redondeo estándar al entero más próximo)
double suelo = Math.floor(valor); // 4.0 (mayor entero menor o igual al valor)
double techo = Math.ceil(valor); // 5.0 (menor entero mayor o igual al valor)

// 4. Comparaciones extremas
int maximo = Math.max(10, 25); // 25
int minimo = Math.min(10, 25); // 10
```

Generación de Números Aleatorios con `Math.random()`

El método `Math.random()` devuelve un número pseudoaleatorio de tipo `double` comprendido en el intervalo semiabierto `[0.0, 1.0)` (incluye el `0.0`, pero nunca alcanza el `1.0`).

Para generar un número entero aleatorio dentro de un rango inclusivo arbitrario `[min, max]`, aplicamos la siguiente fórmula estándar:

```
// Fórmula Universal: (int)(Math.random() * (max - min + 1)) + min
int min = 1;
int max = 6;
int dado = (int)(Math.random() * (max - min + 1)) + min;
System.out.println("Tirada de dado: " + dado);
```

8.2 Cadenas de Texto con la Clase `String`

En Java, un texto **no es un tipo primitivo**, sino un objeto de la clase `java.lang.String`.

La Gran Característica: Inmutabilidad de los Strings

Una vez que un objeto `String` es creado en la memoria Heap, **su contenido jamás puede ser modificado**.

Cualquier método que supuestamente "modifique" un `String` (como `.toUpperCase()`, `.trim()` o `.replace()`) en realidad genera un **nuevo objeto `String`** en memoria con los cambios, dejando el original intacto:

```
String saludo = "hola";
saludo.toUpperCase(); // ¡No modifica 'saludo'! Crea un nuevo String "HOLA" en el aire
System.out.println(saludo); // Imprime: hola

// Para conservar los cambios, debemos reasignar la referencia:
saludo = saludo.toUpperCase();
System.out.println(saludo); // Imprime: HOLA
```

El String Pool y el Gran Error de Comparar Cadenas con `==`

La JVM optimiza el uso de memoria gestionando una zona especial en el Heap llamada **String Constant Pool**.

Cuando creas un literal de texto como `String a = "Java"`, la JVM lo almacena en el pool. Si luego creas `String b`

`= "Java"`, ambas variables apuntan a la misma dirección física de memoria.

Sin embargo, si la cadena se crea en tiempo de ejecución (por ejemplo, leyéndola con `Scanner` o con `new String("Java")`), se aloja en un espacio de memoria distinto:

```
String s1 = "DAM";
String s2 = "DAM";
String s3 = new String("DAM");

System.out.println(s1 == s2); // true (¡Coincidencia fortuita! Apuntan al mismo String Pool)
System.out.println(s1 == s3); // FALSE (¡Error catastrófico! Son dos objetos distintos en Heap)

// REGLA UNIVERSAL E INQUEBRANTABLE EN JAVA:
// Para comparar el contenido de objetos y textos, usa SIEMPRE .equals()
System.out.println(s1.equals(s3)); // TRUE (compara los caracteres reales)
System.out.println(s1.equalsIgnoreCase("dam")); // TRUE (ignora mayúsculas/minúsculas)
```

Prohibido usar `==` con objetos en Java

El operador `==` compara **direcciones de memoria física** (si ambos punteros señalan al mismísimo objeto). Para comparar si dos cadenas tienen el mismo texto alfanumérico, debes usar **siempre** `cadena1.equals(cadena2)`.

Métodos Clave Indispensables de la Clase `String`:

```
String frase = " Aprender Java en DAM es Fantástico! ";

// 1. Longitud del texto
int totalCaracteres = frase.length(); // 39

// 2. Extracción de caracteres por índice (0-based)
char primerCaracter = frase.charAt(2); // 'A'

// 3. Subcadenas (índice inicio inclusivo, índice fin exclusivo)
String sub = frase.substring(2, 10); // "Aprender"

// 4. Limpieza de espacios en blanco
String limpia = frase.strip(); // "Aprender Java en DAM es Fantástico!" (Java 11+)

// 5. Búsqueda de patrones
boolean contieneJava = limpia.contains("Java"); // true
boolean empiezaPor = limpia.startsWith("Aprender"); // true
int posicionDam = limpia.indexOf("DAM"); // 17 (-1 si no existe)

// 6. Reemplazo de caracteres o palabras
String sustituida = limpia.replace("DAM", "1º DAM");

// 7. Comprobación de vacíos (Java 11+)
boolean vacia = "".isEmpty(); // true
boolean enBlanco = " ".isBlank(); // true (solo contiene espacios en blanco)
```

Bloques de Texto (*Text Blocks* - Java 15+)

Para textos multilínea extensos (como consultas SQL, fragmentos HTML o JSON), Java 15 introdujo los bloques de texto delimitados por tres comillas dobles `"""`:

```
String consultaSql = ""
    SELECT id, nombre, apellidos, saldo
    FROM usuarios
    WHERE activo = true
    ORDER BY apellidos ASC;
    "";
```

8.3 Gestión Moderna de Fechas y Tiempo (`java.time`)

Históricamente, Java utilizaba las clases `java.util.Date` y `java.util.Calendar`. Dichas clases fueron declaradas obsoletas debido a graves fallos de diseño: los meses empezaban en 0 (enero era el mes 0), los años empezaban en 1900, eran objetos mutables que provocaban errores de concurrencia y sus métodos tenían nombres confusos.

Desde Java 8, disponemos de la moderna **API `java.time`** (basada en el estándar internacional ISO-8601), que es **completamente inmutable y segura frente a hilos**:

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.temporal.TemporalAdjusters;

public class FechasDemo {
    public static void main(String[] args) {
        // 1. Obtener la fecha y hora actual del sistema
        LocalDate hoy = LocalDate.now();
        LocalTime horaActual = LocalTime.now();
        LocalDateTime momentoActual = LocalDateTime.now();

        // 2. Crear fechas concretas (los meses se indican del 1 al 12 o con el enum
        Month)
        LocalDate inicioCurso = LocalDate.of(2026, 9, 15);

        // 3. Aritmética de fechas (al ser inmutable, devuelve una nueva fecha)
        LocalDate dentroDeDosSemanas = hoy.plusWeeks(2);
        LocalDate haceTresMeses = hoy.minusMonths(3);

        // 4. Cálculo de periodos transcurridos
        Period diferencia = Period.between(inicioCurso, hoy);
        System.out.printf("Tiempo transcurrido: %d meses y %d días%n",
            diferencia.getMonths(), diferencia.getDays());

        // 5. Ajustadores temporales avanzados (ej. último día del mes actual)
        LocalDate finDeMes = hoy.with(TemporalAdjusters.lastDayOfMonth());

        // 6. Formateo y parseo profesional
        DateTimeFormatter formatoEspanol = DateTimeFormatter.ofPattern("dd/MM/yyyy
        HH:mm:ss");
        String fechaTexto = momentoActual.format(formatoEspanol);
        System.out.println("Fecha formateada: " + fechaTexto);

        // Parsear un texto a fecha
        LocalDate fechaParseada = LocalDate.parse("25/12/2026",
            DateTimeFormatter.ofPattern("dd/MM/yyyy"));
    }
}
```

Página 25 de 41

Formateo de Monedas y Números con `DecimalFormat`

Para mostrar importes numéricos con separadores de miles y decimales acordes al formato español:

```
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.util.Locale;

double salario = 2450.60;
DecimalFormat df = new DecimalFormat("#,##0.00 €", new
DecimalFormatSymbols(Locale.of("es", "ES")));
System.out.println(df.format(salario)); // Imprime: 2.450,60 €
```

8.5 Tipos Enumerados (`enum`)

Cuando una variable solo puede tomar un conjunto cerrado, finito y conocido de valores posibles (por ejemplo, los días de la semana, los meses del año, los estados de un pedido o los niveles de dificultad de un juego), utilizar cadenas (`String`) o números enteros (`int`) es propenso a errores tipográficos.

Para resolverlo, Java cuenta con el tipo especial `enum` :

```
public class EnumDemo {
    // Definición del tipo enumerado
    public enum NivelAcceso {
        INVITADO,
        USUARIO,
        ADMINISTRADOR
    }

    public static void main(String[] args) {
        // Uso con seguridad estricta de tipos
        NivelAcceso rolActual = NivelAcceso.USUARIO;

        if (rolActual == NivelAcceso.ADMINISTRADOR) {
            System.out.println("Acceso concedido a la consola de control");
        } else {
            System.out.println("Acceso limitado: Privilegios insuficientes");
        }
    }
}
```

9. Entrada y Salida Estándar por Consola

Todo programa interactivo requiere un canal para recibir datos del usuario y un canal para proyectar sus resultados.

9.1 Salida por Consola: `print()`, `println()` y `printf()`

En Java, la consola se representa mediante el objeto estático `System.out` :

- `System.out.print( ... )` : Muestra el contenido sin salto de línea al final.
- `System.out.println( ... )` : Muestra el contenido y añade automáticamente un salto de línea (`\n`).
- `System.out.printf( ... )` : Permite dar formato avanzado a la salida combinando texto con especificadores de formato (similar a la función `printf` clásica de C).

Tabla Completa de Especificadores de Formato para `printf()`

Especificador	Tipo de Dato	Ejemplo de Entrada	Resultado Formateado
<code>%d</code>	Entero decimal (<code>int</code> , <code>long</code> , <code>byte</code> , <code>short</code>)	<code>printf("%d", 42)</code>	42
<code>%f</code>	Coma flotante (<code>float</code> , <code>double</code>)	<code>printf("%.2f", 3.14159)</code>	3,14 (o 3.14 según Locale)
<code>%s</code>	Cadena de texto (<code>String</code>)	<code>printf("%s", "Java")</code>	Java
<code>%c</code>	Carácter primitivo (<code>char</code>)	<code>printf("%c", 'A')</code>	A
<code>%b</code>	Booleano (<code>boolean</code>)	<code>printf("%b", true)</code>	true
<code>%n</code>	Salto de línea independiente del SO	<code>printf("Hola%nMundo")</code>	Genera salto de línea universal
<code>%%</code>	Carácter de porcentaje literal	<code>printf("%d%%", 100)</code>	100%

Modificadores de Ancho, Alineación y Relleno

Podemos intercalar modificadores entre el `%` y la letra del especificador:

- `%10d` : Justifica a la derecha en un ancho mínimo de 10 columnas.
- `%-10s` : Justifica a la izquierda rellenando con espacios hasta 10 caracteres.
- `%05d` : Rellena con ceros a la izquierda hasta completar 5 dígitos (ej. `00007`).
- `%.2f` : Fija exactamente dos posiciones decimales tras la coma.

```
String producto = "Teclado Mecánico";
int unidades = 4;
double precioUnitario = 79.99;
double total = unidades * precioUnitario;

System.out.println("-----");
System.out.printf("%-20s %5s %10s %10s%n", "PRODUCTO", "CANT", "PRECIO", "TOTAL");
System.out.println("-----");
System.out.printf("%-20s %5d %10.2f %10.2f €%n", producto, unidades, precioUnitario, total);
System.out.println("-----");
```

9.2 Entrada de Datos con la Clase `Scanner`

Para leer datos introducidos por el usuario a través del teclado, la forma estándar en Java es utilizar la clase `java.util.Scanner` , vinculándola al flujo de entrada estándar del sistema `System.in` :

```
import java.util.Scanner;
import java.util.Locale;

public class EntradaDatosDemo {
    public static void main(String[] args) {
        // Creamos una instancia de Scanner asociada al teclado
        Scanner teclado = new Scanner(System.in);

        // Configuramos el punto como separador decimal estándar internacional
        teclado.useLocale(Locale.US);

        System.out.print("Introduce tu nombre: ");
        String nombre = teclado.nextLine(); // Lee una línea completa de texto

        System.out.print("Introduce tu edad: ");
        int edad = teclado.nextInt(); // Lee un número entero

        System.out.print("Introduce tu estatura en metros (ej. 1.78): ");
        double estatura = teclado.nextDouble(); // Lee un número decimal

        System.out.printf("Hola %s, tienes %d años y mides %.2f metros.%n", nombre,
            edad, estatura);

        // Buena práctica: cerrar el Scanner al finalizar el programa
        teclado.close();
    }
}
```

Métodos Principales de Lectura de `Scanner`:

- `nextLine()`: Lee todo el texto hasta que el usuario pulsa Enter (incluye espacios).
- `next()`: Lee únicamente la siguiente palabra (se detiene en el primer espacio en blanco).
- `nextInt()`: Lee el siguiente token interpretándolo como `int`.
- `nextDouble()`: Lee el siguiente token interpretándolo como `double`.
- `nextBoolean()`: Lee `true` o `false`.

9.3 ★ La Trampa del Salto de Línea Residual en `Scanner`

Este es, sin lugar a dudas, **el error número uno que confunde a todos los estudiantes de programación en Java**.

Imagina que ejecutas este código:

```
Scanner sc = new Scanner(System.in);

System.out.print("Introduce tu edad: ");
int edad = sc.nextInt();

System.out.print("Introduce tu nombre completo: ");
String nombre = sc.nextLine(); // ¡SE LO SALTA! No deja escribir nada

System.out.println("Nombre: " + nombre + ", Edad: " + edad);
```

¿Por qué ocurre esto?

Cuando el usuario teclea `25` y pulsa la tecla `Enter`, en el búfer de entrada de la consola se depositan tres caracteres: `'2'`, `'5'` y el carácter de salto de línea `'\n'`.

1. El método `sc.nextInt()` lee únicamente los caracteres numéricos `'2'` y `'5'`, pero **deja el `\n` abandonado en el búfer**.
2. A continuación, `sc.nextLine()` entra en acción buscando una línea hasta encontrar un `\n`.
3. Inmediatamente se encuentra con el `\n` que dejó abandonado `nextInt()`. Como ya ha encontrado un fin de línea, consume el `\n`, devuelve una cadena vacía `""` y el programa continúa sin esperar a que el usuario escriba su nombre.

```
Búfer tras teclear '25' + Enter:
[ '2' | '5' | '\n' ]
  ▲
  nextInt() lee '25' y se detiene aquí. El '\n' permanece en el búfer.

Cuando llega nextLine():
Encuentra inmediatamente el '\n' remanente y termina devolviendo cadena vacía.
```

La Solución: Purgar el Búfer

Siempre que vayas a leer un texto con `nextLine()` después de haber leído un número con `nextInt()`, `nextDouble()` o cualquier otro método que no sea `nextLine()`, debes realizar una **lectura de purga** intermedia para limpiar el salto de línea residual:

```
System.out.print("Introduce tu edad: ");
int edad = sc.nextInt();

sc.nextLine(); // ¡PURGA DEL BÚFER! Consume el '\n' residual abandonado

System.out.print("Introduce tu nombre completo: ");
String nombre = sc.nextLine(); // Ahora sí se detiene y espera la entrada del usuario
```

10. Sentencias de Decisión y Control de Flujo

Por defecto, la Máquina Virtual de Java ejecuta las instrucciones de forma estrictamente secuencial: una línea tras otra, desde la primera sentencia del método `main` hasta la última. Sin embargo, los algoritmos reales requieren tomar bifurcaciones (*branching*) y elegir caminos alternativos en función del estado de las variables y las decisiones del usuario.

10.1 Las Sentencias Condicionales: `if`, `if-else` y Escaleras `if-else if`

La estructura `if` evalúa una expresión lógica de tipo `boolean`. Si la condición resulta ser `true`, ejecuta el bloque de instrucciones delimitado por llaves; si es `false`, omite su ejecución:

```

int edad = 18;

// 1. Estructura if simple
if (edad ≥ 18) {
    System.out.println("Acceso autorizado: Usuario mayor de edad.");
}

// 2. Estructura if-else (Bifurcación completa)
double saldo = 45.50;
double precioArticulo = 60.00;

if (saldo ≥ precioArticulo) {
    saldo -= precioArticulo;
    System.out.println("Compra realizada con éxito. Saldo restante: " + saldo);
} else {
    double falta = precioArticulo - saldo;
    System.out.printf("Saldo insuficiente. Faltan %.2f € para completar la compra.%n",
        falta);
}

```

Escalera de Condiciones Múltiples: `if - else if - else`

Cuando debemos clasificar una situación entre múltiples rangos excluyentes:

```

double nota = 7.8;

if (nota < 0.0 || nota > 10.0) {
    System.out.println("Error: Calificación fuera de rango válido (0-10).");
} else if (nota < 5.0) {
    System.out.println("Suspenso");
} else if (nota < 6.5) {
    System.out.println("Aprobado");
} else if (nota < 8.5) {
    System.out.println("Notable");
} else if (nota < 10.0) {
    System.out.println("Sobresaliente");
} else {
    System.out.println("Matrícula de Honor");
}

```

El Peligro Mortal de Omitir las Llaves `{}`

En Java, si una cláusula `if` o `else` contiene una única sentencia, las llaves `{}` son opcionales según la gramática del compilador. **Sin embargo, en el desarrollo de software profesional está terminantemente prohibido omitirlas.**

Observa este clásico desastre:

```
// Código engañoso sin llaves:
if (usuarioEsAdmin)
    registrarAcceso();
    borrarBaseDeDatos(); // ¡PELIGRO! Esta línea se ejecuta SIEMPRE, sea admin o
no
```

La sangría visual engaña al ojo humano, pero el compilador solo vincula la primera sentencia (`registrarAcceso()`) al `if`. La segunda línea no forma parte del condicional. El infame fallo de seguridad SSL de Apple en 2014 (*goto fail;*) que costó millones de dólares se debió exactamente a omitir las llaves. **Pon siempre llaves `{}` en todos tus bloques de control.**

10.2 La Sentencia `switch` Clásica

Cuando una misma variable debe compararse frente a múltiples valores constantes discretos, encadenar decenas de `if-else if` resulta farragoso y poco eficiente. Para estos escenarios, Java proporciona la sentencia `switch`:

Tipos de Datos Admitidos por `switch`:

Java permite utilizar `switch` únicamente con los siguientes tipos:

- Primitivos enteros pequeños: `byte`, `short`, `char`, `int`.
- Sus correspondientes clases envoltorio: `Byte`, `Short`, `Character`, `Integer`.
- Cadenas de texto `String` (admitido desde Java 7).
- Tipos enumerados `enum` (admitido desde Java 5).

Tipos NO admitidos en `switch`

En Java **está prohibido** utilizar `switch` con variables de tipo `long`, `float`, `double` y `boolean`. Para números decimales o rangos continuos debes emplear `if-else`.

```
int diaSemana = 3;
String nombreDia;

switch (diaSemana) {
    case 1:
        nombreDia = "Lunes";
        break; // Detiene la ejecución y sale del bloque switch
    case 2:
        nombreDia = "Martes";
        break;
    case 3:
        nombreDia = "Miércoles";
        break;
    case 4:
        nombreDia = "Jueves";
        break;
    case 5:
        nombreDia = "Viernes";
        break;
    case 6:
        nombreDia = "Sábado";
        break;
    case 7:
        nombreDia = "Domingo";
        break;
    default: // Se ejecuta si ningún caso anterior coincide
        nombreDia = "Día inválido";
        break;
}

System.out.println("Hoy es: " + nombreDia);
```

El Fenómeno de Caída Secuencial (*Fall-Through*)

En el `switch` clásico heredado del lenguaje C, si olvidas colocar la sentencia `break` al final de un `case`, la JVM continuará ejecutando las instrucciones de los siguientes `case` de manera secuencial, sin importar si sus condiciones coinciden o no:

- **Caída Involuntaria (Bug común):** Si olvidas el `break` en `case 1:`, al valer 1 ejecutará el código del caso 1 y también el código del caso 2.
- **Caída Intencionada (Técnica útil):** Podemos agrupar deliberadamente múltiples casos para que compartan la misma lógica:

```

int mes = 2;
int anio = 2024;
int diasEnElMes;

switch (mes) {
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        diasEnElMes = 31;
        break;
    case 4: case 6: case 9: case 11:
        diasEnElMes = 30;
        break;
    case 2:
        // Comprobación de año bisiesto
        if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
            diasEnElMes = 29;
        } else {
            diasEnElMes = 28;
        }
        break;
    default:
        diasEnElMes = 0;
        System.out.println("Mes erróneo");
        break;
}

```

10.3 El `switch` Moderno: Expresiones `switch` y Flechas → (Java 14+)

Java 14 estandarizó una revolución en la sintaxis de control: las **expresiones `switch`** con operadores de flecha (→). Esta sintaxis moderna resuelve todos los problemas históricos del `switch` clásico:

1. **Elimina el peligro de *fall-through***: No se requiere escribir `break`. Solo se ejecuta la instrucción situada tras la flecha.
2. **Agrupación limpia por comas**: Múltiples etiquetas pueden separarse sencillamente con comas (`case 1, 2, 3 →`).
3. **Funciona como expresión**: Puede devolver un valor directamente para asignarlo a una variable.
4. **Comprobación exhaustiva**: El compilador verifica que todos los casos posibles estén cubiertos (o que exista una cláusula `default`).

```

int dia = 6;

// 1. Switch moderno como sentencia de acción (sin break, sin fall-through)
switch (dia) {
    case 1, 2, 3, 4, 5 → System.out.println("Día laborable: ¡A trabajar!");
    case 6, 7          → System.out.println("Fin de semana: ¡A descansar!");
    default            → System.out.println("Número de día inválido.");
}

// 2. Switch moderno como EXPRESIÓN (asigna el resultado directamente)
String tipoDeDia = switch (dia) {
    case 1, 2, 3, 4, 5 → "Laborable";
    case 6, 7          → "Festivo";
    default            → "Desconocido";
};

System.out.println("El día seleccionado es de tipo: " + tipoDeDia);

```

La Palabra Clave Contextual `yield`

Si un caso de una expresión `switch` requiere un bloque de varias líneas de código para realizar cálculos antes de devolver el valor, se utilizan llaves `{ ... }` y la palabra clave `yield` para emitir el resultado:

```
int nivelUsuario = 2;

double descuento = switch (nivelUsuario) {
    case 1 → 0.05;
    case 2 → {
        double bonusFidelidad = 0.03;
        double base = 0.10;
        yield base + bonusFidelidad; // Emite 0.13 como valor resultante
    }
    case 3 → 0.20;
    default → {
        System.out.println("Nivel no reconocido, asignando descuento cero.");
        yield 0.0;
    }
};
```

11. Tipología Práctica de Errores en Java y Depuración con IntelliJ IDEA

El desarrollo de software no consiste únicamente en escribir código; una gran parte de la labor diaria de un desarrollador es diagnosticar, aislar y corregir errores (*debugging*).

En Java, los errores se dividen en tres grandes categorías según la etapa en la que se manifiestan:

Clasificación de Errores en Java

- └ 1. Errores de Compilación (Sintácticos / Semánticos) → Detectados por `javac` / IntelliJ
- └ 2. Errores en Tiempo de Ejecución (Excepciones) → Detienen la JVM abruptamente
- └ 3. Errores Lógicos (Bugs de Algoritmo) → El programa corre, pero el resultado es erróneo

11.1 Errores de Compilación (*Compile-time Errors*)

Ocurren cuando el código viola las reglas sintácticas o semánticas del lenguaje Java. El compilador `javac` no genera el archivo `.class` y el programa no puede ser ejecutado.

IntelliJ IDEA detecta estos errores en tiempo real conforme tecleas, subrayando el código con una línea ondulada roja y ofreciendo soluciones contextuales mediante el atajo universal `Alt + Enter`:

```
// Ejemplos típicos:
int numero = "Hola";           // Incompatibilidad de tipos: String no puede asignarse a
int                               int
int total                       // Falta el punto y coma final
System.out.println(valor);     // Variable 'valor' no declarada o fuera de ámbito
int x; int y = x + 1;          // Variable local 'x' no inicializada
```

11.2 Errores en Tiempo de Ejecución (*Runtime Errors / Excepciones*)

El programa compila limpiamente a Bytecode, pero durante la ejecución ocurre una situación anómala o imprevista que la Máquina Virtual no puede resolver por sí misma. Como consecuencia, el hilo principal (`Thread "main"`) se estrella, aborta el programa y escupe por consola una traza de error denominada **Stack Trace**.

Las 4 Excepciones Clásicas de la Unidad 2:

1. `java.lang.ArithmeticException: / by zero` :

Ocurre al intentar dividir un número entero por cero (`10 / 0`).

(Curiosidad: en números decimales, `10.0 / 0.0` no lanza excepción; devuelve `Infinity` según el estándar IEEE 754).

2. `java.lang.NullPointerException (NPE)`:

El error más famoso del ecosistema Java. Ocurre cuando intentas invocar un método o acceder a un atributo a través de una variable de referencia que vale `null` (no apunta a ningún objeto real en la memoria Heap):

```
String nombre = null;
System.out.println(nombre.length()); // ¡Lanza NullPointerException!
```

3. `java.util.InputMismatchException` :

Ocurre cuando la clase `Scanner` espera recibir un token de un tipo específico y el usuario teclea algo incompatible:

```
System.out.print("Introduce tu edad: ");
int edad = sc.nextInt(); // El usuario teclea "veinte" → ¡InputMismatchException!
```

4. `java.lang.NumberFormatException` :

Ocurre al intentar parsear una cadena alfanumérica mediante `Integer.parseInt("abc")` o `Double.parseDouble("12,5")` (con coma en vez de punto en configuración US).

Cómo Leer e Interpretar un Stack Trace

Cuando veas un mensaje de error rojo en la consola de IntelliJ IDEA, no te asustes: es un mapa exacto que te guía directamente hacia el culpable:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at es.iesperenxisa.primerospasos.Calculadora.dividir(Calculadora.java:24)
    at es.iesperenxisa.primerospasos.Calculadora.main(Calculadora.java:12)
```

Pasos para interpretarlo:

1. **Primera línea:** Nombre de la excepción y causa (`ArithmeticException: / by zero`).
2. **Segunda línea:** Método donde ocurrió la catástrofe (`dividir`), archivo (`Calculadora.java`) y **número exacto de línea** (`24`). En IntelliJ IDEA, ese texto es un hipervínculo azul: ¡haz clic sobre él y el editor te llevará directamente a la línea que falló!

11.3 Errores Lógicos (*Bugs*)

Son los más traicioneros y difíciles de detectar. El programa compila sin advertencias y se ejecuta sin lanzar ninguna excepción, pero el resultado que arroja es matemáticamente o lógicamente incorrecto.

```
// Ejemplo de Bug Lógico:
double nota1 = 8.0;
double nota2 = 6.0;

// Intención: calcular la media aritmética (8 + 6) / 2 = 7.0
// Error: por precedencia de operadores, primero divide 6.0 / 2 = 3.0, y luego suma 8.0
// = 11.0
double media = nota1 + nota2 / 2; // ¡Error lógico! Faltan los paréntesis: (nota1 +
// nota2) / 2
```

11.4 Depuración Profesional (*Debugging*) con IntelliJ IDEA

Para cazar errores lógicos, los principiantes suelen llenar el código de `System.out.println()`. Los ingenieros de software profesionales utilizan el **Depurador Integrado** (*Debugger*).

El depurador te permite “congelar el tiempo”, detener la ejecución de tu programa en una línea exacta y avanzar paso a paso mientras inspeccionas el valor vivo de cada variable en la memoria RAM:

Guía Rápida de Depuración en IntelliJ IDEA:

1. **Poner un Punto de Interrupción (*Breakpoint*)**: Haz clic con el ratón en el margen gris izquierdo (*gutter*) justo al lado del número de línea donde deseas que el programa se pause. Aparecerá un círculo rojo sólido.
2. **Lanzar en Modo Debug**: En lugar de pulsar el botón verde de Play normal, pulsa el icono de la **cucaracha verde** (*Debug*) o utiliza el atajo `Shift + F9`.
3. **Inspección en Pausa**: Cuando la ejecución alcance tu breakpoint, la línea se resaltará en azul y el programa quedará congelado. En la parte inferior se abrirá el panel **Debug**:
 - Pestaña **Variables**: Muestra el valor en tiempo real de cada variable en el Stack y en el Heap.
4. **Controles de Avance Paso a Paso**:
 - **Step Over (F8)**: Ejecuta la línea actual y avanza a la siguiente línea del mismo método.
 - **Step Into (F7)**: Si la línea actual contiene una llamada a un método propio, se adentra en su código para ver qué hace por dentro.
 - **Step Out (Shift + F8)**: Sale del método actual y regresa al llamador.
 - **Resume Program (F9)**: Reanuda la ejecución continua hasta encontrar el siguiente breakpoint.
 - **Stop (Ctrl + F2)**: Detiene la sesión de depuración.
5. **Evaluar Expresiones al Vuelo (*Evaluate Expression* - Alt + F8)**: Puedes abrir una ventana flotante para escribir cualquier cálculo de Java y evaluarlo al instante utilizando los valores vivos de las variables congeladas.

12. ★ Be the Code: Desafíos de Traza Mental y Memoria

Para ser un programador excepcional debes ser capaz de “ser el código”: proyectar mentalmente el estado de la memoria RAM y predecir con exactitud qué valor adoptará cada variable en cada ciclo de instrucción.

Desafío 1: El Laberinto de los Incrementos y la Precedencia

Analiza el siguiente fragmento de código sin ejecutarlo en la máquina. ¿Qué valor imprimirán las variables `a`, `b` y `c`?

```
int a = 10;
int b = 20;
int c = a++ + ++b * 2;
```

Traza de Ejecución Paso a Paso:

1. Inicialmente: `a = 10`, `b = 20`.
2. Evaluamos la expresión de derecha a izquierda respetando la precedencia:
 - El operador `++` prefijo sobre `b` (`++b`) tiene máxima prioridad: `b` se incrementa inmediatamente de 20 a **21**, y ese valor `21` se utiliza en la expresión.
 - Operador multiplicativo `*`: se multiplica `21 * 2 = 42`.
 - El operador `++` postfijo sobre `a` (`a++`): se utiliza en la suma su valor actual (**10**), y justo después de ser leído, `a` se incrementa en memoria a **11**.
 - Operador aditivo `+`: se suma `10 + 42 = 52`.
3. Asignación a `c`: `c` recibe el valor **52**.

Variable	Valor Final en Memoria
<code>a</code>	11
<code>b</code>	21
<code>c</code>	52

Desafío 2: Algoritmo de Descomposición de Cifras con `/` y `%`

Supongamos que un usuario introduce el número entero `7429`. Diseña mentalmente el algoritmo aritmético para aislar de forma independiente las unidades, decenas, centenas y millares:

```
int numero = 7429;

int unidades = numero % 10;           // 7429 % 10 = 9
int decenas  = (numero / 10) % 10;    // 7429 / 10 = 742 → 742 % 10 = 2
int centenas = (numero / 100) % 10;   // 7429 / 100 = 74 → 74 % 10 = 4
int millares = numero / 1000;         // 7429 / 1000 = 7
```

Principio algorítmico universal:

- El operador `% 10` extrae siempre el último dígito menos significativo de un número decimal.
- La división entera `/ 10` desplaza el número hacia la derecha, descartando el último dígito.

Desafío 3: El String Pool y el Mapa de Memoria Stack vs Heap

Observa detenidamente este código y predice qué imprimirá cada una de las 4 comparaciones:

```
String texto1 = "Java";
String texto2 = "Java";
String texto3 = new String("Java");
String texto4 = "Ja" + "va"; // Expresión constante evaluada por el compilador

System.out.println(texto1 == texto2); // ¿true o false?
System.out.println(texto1 == texto3); // ¿true o false?
System.out.println(texto1 == texto4); // ¿true o false?
System.out.println(texto1.equals(texto3)); // ¿true o false?
```

Solución Razonada con Arquitectura de Memoria:

1. `texto1 == texto2` devuelve `true`: Ambas variables apuntan a la misma entrada compartida en el *String Constant Pool*.
2. `texto1 == texto3` devuelve `false`: La palabra clave `new` fuerza la creación de un nuevo objeto independiente en el Heap general, con una dirección física de memoria distinta.
3. `texto1 == texto4` devuelve `true`: Como `"Ja"` y `"va"` son literales constantes conocidos en tiempo de compilación, el compilador `javac` optimiza la concatenación y la resuelve como `"Java"`, reutilizando la referencia del String Pool.
4. `texto1.equals(texto3)` devuelve `true`: El método `.equals()` no compara direcciones de memoria física, sino la secuencia exacta de caracteres alfanuméricos (`'J'`, `'a'`, `'v'`, `'a'`).

13. ¡No Hay Preguntas Tontas!

1. ¿Por qué Java tiene tantos tipos enteros (`byte`, `short`, `int`, `long`) si lenguajes modernos como JavaScript o Python usan uno solo?

Porque Java fue diseñado desde su origen para sistemas de misión crítica, servidores de alto rendimiento y dispositivos embebidos. Si tienes un array de 10 millones de números que solo van del 0 al 100:

- Con `int` (4 bytes por número) consumes **40 MB** de RAM.
- Con `byte` (1 byte por número) consumes solo **10 MB** de RAM (un 75% menos de memoria).

En aplicaciones a gran escala o microservicios que procesan millones de peticiones por segundo, la gestión milimétrica de la memoria y la caché del procesador marca la diferencia entre un sistema fluido y uno colapsado.

2. ¿Por qué `Scanner` me lanza un error si escribo `1.75` con punto en lugar de coma?

Porque la clase `Scanner` por defecto utiliza la **configuración regional (Locale) del sistema operativo** de tu máquina. En los sistemas configurados en España u otros países europeos, el separador decimal estándar oficial es la coma (`,`), por lo que espera `1,75`. Si introduces `1.75` con punto, lanza `InputMismatchException`. Para forzar a que tu programa utilice siempre el punto decimal estándar internacional, añade la siguiente línea tras instanciar el `Scanner`:

```
sc.useLocale(Locale.US);
```

3. ¿Por qué `String` no es un tipo primitivo si lo usamos constantemente como si lo fuera?

Porque un texto no tiene un tamaño fijo en bytes en la memoria física: puede tener cero caracteres, una palabra o una enciclopedia completa de millones de caracteres. Los tipos primitivos tienen un tamaño estrictamente inmutable

en hardware (8, 16, 32 o 64 bits). Por ello, `String` es un objeto complejo alojado en el Heap que internamente encapsula un array de bytes y ofrece decenas de métodos utilitarios.

4. ¿Es `var` una señal de pereza o mala práctica de programación?

Rotundamente no, siempre que se use con criterio profesional. `var` fue incorporado en Java 10 para eliminar la redundancia innecesaria que ensuciaba el código clásico:

```
Map<String, List<Pedido>> mapa = new HashMap<String, List<Pedido>>();
```

Con `var`, la línea se reduce limpiamente a:

```
var mapa = new HashMap<String, List<Pedido>>();
```

No se pierde ninguna información de tipos (el compilador sabe exactamente qué es) y el código gana en claridad y limpieza visual.

5. ¿Por qué Java mantiene `float` y `double` si tienen errores de redondeo y en banca se usa `BigDecimal`?

Porque `double` y `float` están implementados directamente en la circuitería de hardware del procesador (la FPU, *Floating Point Unit*), lo que permite ejecutar **miles de millones de cálculos por segundo**. En videojuegos 3D, simulaciones físicas, renderizado gráfico e inteligencia artificial, la velocidad extrema es prioritaria y una diferencia en el decimosexto decimal es irrelevante. En cambio, `BigDecimal` es una clase de software: sus operaciones son miles de veces más lentas, pero garantizan precisión matemática absoluta. Cada herramienta tiene su propósito.

6. ¿Por qué un `char` puede sumarse con un número `int` sin que el compilador se queje?

Porque en la arquitectura de Java, un `char` es en realidad un número entero sin signo de 16 bits que almacena el código numérico Unicode del carácter. Por ejemplo, el carácter `'A'` equivale internamente al número `65`. Al hacer `'A' + 1`, Java promociona el `char` a `int` y efectúa la suma matemática `65 + 1 = 66`. Si luego haces un casting explícito `(char) 66`, obtendrás el carácter `'B'`.

7. ¿Qué diferencia real hay entre `&&` y `&`, o `||` y `|` si ambos dan el mismo resultado lógico?

La diferencia radica en la **evaluación en cortocircuito**:

`&&` y `||` se detienen tan pronto como conocen el resultado definitivo, ahorrando tiempo de CPU y permitiendo programar barreras de protección contra excepciones como `NullPointerException` o división por cero. En cambio, `&` y `|` evalúan obligatoriamente ambos lados de la expresión, sin importar lo que haya devuelto el primero. En lógica condicional ordinaria, **debes usar siempre `&&` y `||`**.

14. Resumen de la Unidad y Enlace con la Unidad 3

Lo que hemos conquistado en esta Unidad:

1. **Ecosistema:** La JVM, el compilador `javac`, el Bytecode independiente de plataforma y el IDE IntelliJ IDEA con soporte para las últimas versiones LTS (Java 21 / Java 25 LTS).
2. **Anatomía:** Estructura de clases, paquetes de dominio inverso, imports y el método canónico `public static void main(String[] args)`.
3. **Tipos y Memoria:** Los 8 tipos primitivos, variables vs constantes (`final`), inferencia con `var`, y el desbordamiento en complemento a dos.
4. **Conversiones:** Promoción implícita (*widening*), casting explícito (*narrowing*), parseo de texto y clases envoltorio (*wrappers*) con autoboxing.

5. **Operadores:** Aritmética entera vs real, módulo `%`, incrementos pre/post, operadores relacionales y operadores lógicos con evaluación en cortocircuito.
6. **Biblioteca Estándar:** Cálculos y aleatoriedad con `Math`, inmutabilidad y String Pool con `String`, precisión monetaria con `BigDecimal`, tipos cerrados con `enum` y fechas modernas con `java.time`.
7. **Entrada/Salida:** Formateo con `printf`, lectura por teclado con `Scanner` y la técnica de purga del salto de línea residual.
8. **Decisiones:** Ramificaciones lógicas con `if-else` y la potencia del `switch` moderno con sintaxis de flecha `->` y `yield`.
9. **Depuración:** Diagnóstico de excepciones y rastreo paso a paso con puntos de interrupción en IntelliJ IDEA.

¿Hacia dónde vamos en la Unidad 3?

Hasta ahora nuestros programas han tomado decisiones y ejecutado instrucciones de arriba hacia abajo una única vez. Pero el verdadero poder de la computación reside en la **automatización repetitiva masiva**:

- En la **Unidad 3: Estructuras de Control Iterativas, Bucles y Métodos**, aprenderemos a repetir bloques de código miles o millones de veces con bucles `while`, `do-while` y `for`.
- Descubriremos cómo empaquetar conjuntos de datos en **arrays** unidimensionales y matrices bidimensionales.
- Dividiremos programas complejos en bloques reutilizables mediante **métodos estáticos** modulares.
- Blindaremos nuestras aplicaciones ante fallos del usuario implementando una gestión profesional de excepciones con bloques `try-catch-finally`.

15. Relación Curricular Oficial (CFGS DAM - Módulo 0485)

Esta unidad cubre de manera directa y exhaustiva los siguientes **Resultados de Aprendizaje (RA)** y **Criterios de Evaluación (CE)** establecidos en el currículo oficial del ciclo formativo:

- **RA 1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**
 - **CE 1.a:** Se ha identificado la estructura general de un programa y sus bloques constituyentes.
 - **CE 1.b:** Se han utilizado entornos integrados de desarrollo (IntelliJ IDEA) para la edición, compilación y ejecución de programas.
 - **CE 1.c:** Se han identificado los diferentes tipos de datos primitivos disponibles en el lenguaje.
 - **CE 1.d:** Se han declarado y utilizado variables, constantes y literales respetando las normas léxicas y de estilo.
 - **CE 1.e:** Se han utilizado operadores aritméticos, relacionales y lógicos respetando la precedencia de operadores.
 - **CE 1.f:** Se han clasificado y documentado los programas utilizando comentarios descriptivos de línea, bloque y Javadoc.
- **RA 4: Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.**
 - **CE 4.a:** Se ha reconocido la sintaxis, estructura y componentes de una clase en Java.
 - **CE 4.e:** Se han utilizado métodos estándar de la biblioteca del lenguaje (`Math`, `String`, `BigDecimal`, `LocalDate`).

- **RA 5: Realiza operaciones de entrada y salida de información utilizando la consola y clases estándar.**
 - **CE 5.a:** Se han utilizado los flujos estándar de entrada y salida (`System.out` , `System.in`).
 - **CE 5.b:** Se ha utilizado la clase `Scanner` para la lectura interactiva de datos por teclado, gestionando delimitadores y configuraciones regionales.
 - **CE 5.c:** Se ha aplicado formato avanzado a los datos de salida mediante cadenas de formato y `printf` .

16. Boletines de Ejercicios y Retos Prácticos

Para afianzar cada uno de los conceptos aprendidos, dispones de una batería progresiva de ejercicios prácticos especialmente diseñados para este nivel:

1. **[Boletín 2.1 — Inicial](#)**: 13 ejercicios guiados paso a paso sobre declaración de variables, cálculos aritméticos elementales, conversiones de tipos, entrada/salida formateada por consola y evaluación de condiciones lógicas.
 - Consulta también el **[Boletín Inicial \(Resuelto\)](#)** con explicaciones y código fuente completo en Java 21 LTS.
2. **[Boletín 2.2 — Intermedio](#)**: 13 ejercicios aplicados sobre sentencias condicionales `if-else` y `switch` , manipulación avanzada de cadenas con `String` , funciones de `Math` , tipos `enum` , cálculo temporal con `java.time` y geometría computacional.
 - Consulta también el **[Boletín Intermedio \(Resuelto\)](#)** con soluciones detalladas.
3. **[Boletín 2.3 — Retos y Extras](#)**: 6 grandes desafíos algorítmicos competitivos (verificación oficial de DNI, año bisiesto gregoriano, simulación probabilística de dados, factoriales astronómicos con `BigInteger` , algoritmo voraz de cambio monetario exacto y resolución de ecuaciones de segundo grado con análisis de discriminante).